A Boolean linear-time planarity test

MohammadTaghi Hajiaghayi Mahdi JafariRaviz

November 2026

Abstract

We give a deterministic program that decides in time $O(N + M + 1)$ whether a finite simple graph with N vertices and M edges is planar. It returns only the answer, not an embedding. The program is a left-right planarity test in the tradition of de Fraysseix and Rosenstiehl. The note proves everything it uses: the correctness of the constraint solver, and the left-right characterization of planarity itself. The only fact about planar graphs that we use is Euler's edge bound. The test supplies the planarity primitive of a companion note on extracting crossings, which replaces the two paths algorithm of Shiloach in Graph Minors XIII.

Introduction

All graphs in this note are finite and simple. A *plane embedding* of a graph places its vertices at distinct points of the plane and draws each edge as an injective continuous arc between the points of its two ends. Two edge images meet only at common ends, and no edge image contains a vertex point other than its own ends. A graph is *planar* if it has a plane embedding. Part IV, Section 6 also treats embeddings given combinatorially, by rotation systems.

Theorem. *There is a single deterministic word-RAM program which, given the adjacency lists of a graph with N vertices and M edges, returns true exactly when the graph has a plane embedding. Its time and space are $O(N + M + 1)$. The program need not return an embedding.*

How the test works. Run a depth-first search. In a plane embedding, every back edge leaves the search tree either on the left or on the right of the tree path it closes. De Fraysseix and Rosenstiehl showed that a graph is planar if and only if its back edges can be split into left and right ones so that certain sets of back edges get one side and certain pairs of sets get opposite sides [dFR82, dFR85]. In the form of de Fraysseix, Ossona de Mendez and Rosenstiehl [dFOMR06] (Theorem 2 on page 8 of the arXiv version), and in Brandes's description of the left-right planarity test [Br09, Theorem 5, page 10], these are constraints on two colors L and R of the back edges. Part I of this note decides these constraints in one pass over the search tree. Each vertex hands its parent a compact normal form of the constraints found below it: a stack of blocks, each block made of two lists of back edges. We add one extra condition, weak alignment, which keeps this normal form simple. It does not change whether the constraints can be satisfied (Part III). Brandes uses a related alignment of the partition [Br09, Lemma 6, page 11].

The program uses depth-first search, integer bucket sorting and linked lists. No planarity test is assumed. We prove every characterization and every property of the data structure that we use. The cited works are background, not premises of the theorem. The only fact about planar graphs that we use is Euler's bound: a simple planar graph with $N \geq 3$ vertices has at most $3N - 6$ edges. It is used in Part IV, Section 7c. Part IV, Section 6 gives a second proof of the same step for rotation systems, which uses Euler's formula instead.

Where the test is used. The companion note [HJ26] gives an algorithm that extracts a crossing of a cycle in time $O(nm)$. It calls a Boolean linear-time planarity test, which it states as Fact P. That algorithm replaces Shiloach's two paths algorithm [Sh80] in (8.1) of Graph Minors XIII [GM13, page 93]. Linear-time planarity testing goes back to Hopcroft and Tarjan [HT74]. In the Lean formalization of the Graph Minors series, the test of this note proves Fact P, so the main result of Graph Minors XIII is proved there without assuming a planarity algorithm (Part VI).

Outline. Part I gives the algorithm. Part II proves that it decides exactly the constraints stated in Part I, Section 2. Part III shows that weak alignment does not change satisfiability. Part IV proves that the constraints can be satisfied exactly when the graph is planar. Part V bounds the running time on a word RAM, assembles the correctness proof, explains the use in the crossing extractor, and describes tests of the reference implementation `planarity_linear.py`. Part VI describes the Lean formalization.

Part I

The exact algorithm

This part gives the whole algorithm. Section 1 runs a depth-first search and computes for every edge a key, which fixes the order in which edges are processed. Section 2 states the constraints on two-colorings of the back edges that the algorithm decides. Sections 3 to 6 give the data structure and the procedure that decides them.

1 DFS definitions and preprocessing

We run an ordinary depth-first search of each component. We orient tree edges from parent to child and all other edges from descendant to ancestor. Let $h(v)$ be the depth of v, with root depth zero, and let $\mathrm{subtree}(v)$ be the set of descendants of v, including v. All non-tree edges have comparable ends: if an undirected edge joined two different branches of the search, then scanning it from the first branch would discover the other end before that branch is finished, a contradiction. We call the non-tree edges *back edges*.

For a back edge $b = (x, u)$, put $d(b) = h(u)$. The *outgoing edges* of a vertex v are the tree edges from v to its children and the back edges from v. For an outgoing edge e of v, define

$$F(e) = \{e\}, \qquad \text{if } e \text{ is a back edge,}$$
$$F(e) = \{\text{back edges } (x,u) : \ x \in \mathrm{subtree}(w),\ h(u) < h(v)\}, \qquad \text{if } e = (v,w) \text{ is a tree edge.}$$

These are the *strict returns* of e. Define

$$\mathrm{low}(e) = \min(\{h(v)\} \cup \{d(b) : b \in F(e)\}),$$
$$\mathrm{thick}(e) = 1 \text{ if } F(e) \text{ has two different } d\text{-values, and 0 otherwise,}$$
$$\mathrm{key}(e) = 2\,\mathrm{low}(e) + \mathrm{thick}(e).$$

We also call a thick edge *chordal*, and write $\mathrm{chordal}(e) = \mathrm{thick}(e)$. We order the outgoing edges of every vertex by increasing key, breaking ties arbitrarily. Thus a lower lowpoint comes first, and at an equal lowpoint a thin edge comes before a thick one. An edge with empty $F(e)$ has lowpoint $h(v)$, and the constraint solver ignores it.

Computing the lowpoints. We compute these values without listing the sets $F(e)$. We keep a pair $(\text{low}, \text{low2})$ for every edge. For a tree edge with source depth c, we initialize both values to c. For a back edge (v, u), we initialize them to $(h(u), h(v))$. When an outgoing edge f of v is completed, we insert $\text{low}(f)$ and $\text{low2}(f)$ into the pair of the incoming tree edge of v, by the following operation.

```
1: function INSERT(k)
2:     if k < low then
3:         low2 ← low
4:         low ← k
5:     else if low < k and k < low2 then
6:         low2 ← k
```

This keeps the two smallest distinct values, with the source depth as a repeated upper sentinel. Values at or above the sentinel are ignored. To prove the recurrence for a tree edge (u, v), split the back edges with tail in subtree(v) into the back edges that leave v and, for each child w of v, the back edges with tail in subtree(w). These parts belong to the outgoing edges of v. The two smallest distinct depths below the sentinel in a union are among the two smallest distinct depths below it in the parts: a third distinct value of one part already has two smaller values in that part. This proves the recurrence by induction in the postorder of the search. The correct thick bit is

$$\text{thick}(e) = \big[\text{low2}(e) < h(\text{source of } e)\big].$$

Returns to the source of e itself cannot set this bit, as required, because they are not strict returns.

Sorting by key. We bucket-sort all edge records by their integer keys in one global pass, and then append them to the outgoing lists of their sources. Keys lie in $[0, 2N]$, so one global scan of that range is enough. We do not allocate and scan a bucket array of length N separately for each vertex.

The search with arrays. A depth-first search with arrays needs no edge dictionary. When it scans an incidence v–w:

- if w is unvisited, w becomes a tree child of v;
- otherwise, if w is the parent of v, we skip this reverse tree incidence;
- otherwise, we create a back-edge record exactly when $h(w) < h(v)$.

The other incidences are ignored. Since there are no cross edges, every undirected edge gets exactly one oriented record. Since the graph is simple, skipping by parent vertex skips only the tree edge. Explicit frames $(\text{vertex}, \text{next adjacency index})$ produce the postorder of the search without recursion.

2 The constraints the algorithm decides

Give every back edge one of two colors, L and R. For distinct outgoing edges e, f of a vertex, write

$$I(e, f) = \{b \in F(e) : \ d(b) > \text{low}(f)\}.$$

The *fork constraints* are:

- the set $I(e, f)$ is monochromatic;
- the set $I(f, e)$ is monochromatic;
- if both sets are nonempty, their colors are different.

An empty or one-element set is monochromatic and adds no equality. These are the constraints of the left-right criterion: $I(e, f)$ is the set Interlaced(e, f) of [dFOMR06] (Theorem 2 on page 8 of the arXiv version).

For a tree edge $p = (u, v)$, define its *weak returns*

$$W(p) = \{\text{back edges } (x, t) : \ x \in \text{subtree}(v), \ h(t) \leq h(u)\}.$$

Equivalently, $W(p)$ is the union of the sets $F(e)$ over the outgoing edges e of v. We also require that all members of minimum depth of a nonempty set $W(p)$ have the same color. We call this *weak alignment*. The source u is included in the definition, so alignment is imposed before the returns to u are removed.

Parts II to IV prove that these constraints, including weak alignment, can be satisfied exactly when the graph is planar.

3 Representation

A *list* consists of nodes for back edges, with a first pointer and a last pointer. A node contains its depth key d and a next pointer. Lists are ordered by nonincreasing d. Concatenation is destructive: we connect the old last node to the new first node, update the last pointer, and empty the header of the list that was appended. This costs $O(1)$, whatever the number of nodes.

A *block* (L, R) consists of two such lists, not both empty. It means that each list is monochromatic and, if both lists are nonempty, their colors differ. Different blocks can be flipped independently.

A nonempty *summary* is a triple (a, A, P), where

- the number a is the least live depth;
- the *anchor* A is a nonempty list that contains *all* nodes of depth a;
- the *block stack* P is a stack of blocks, and its blocks contain every other live node exactly once.

The anchor is one independent monochromatic class. Every node in P has depth strictly greater than a. For two blocks of the stack, every depth in the upper block is at least every depth in the lower block. Equality is allowed.

For a nonempty list, its maximum is the key of its first node and its minimum is the key of its last node. Let $\max(\varnothing) = -1$. The maximum and minimum of a block use the maxima and minima of its nonempty lists, so both are $O(1)$ operations. The stack has a top pointer, and each block has a pointer to the block below it. The anchor is stored separately, so we never need to delete at the bottom of a singly linked stack.

4 Merge pseudocode

The operation MERGE transfers ownership destructively: it consumes T and returns the updated S.

```
1: function MERGE(S, T)
2:     if T is empty then return S
3:     if S is empty then return T
4:     a ← S.a
5:     b ← T.a                          ▷ the edge order guarantees a ≤ b
6:     R ← empty list
7:     while T.P is not empty do
8:         Q ← POP(T.P)
```

```
        if Q.L and Q.R are both nonempty then reject the graph
        append the nonempty side of Q to R
    if b = a then
        append T.A to S.A
    else
        append T.A to R
    L ← empty list
    while S.P is not empty and max(TOP(S.P)) > b do
        Q ← POP(S.P)
        if max(Q.R) > b then swap Q.L and Q.R
        if max(Q.R) > b then reject the graph
        append Q.L to L
        append Q.R to R
    if L or R is nonempty then push (L, R) onto S.P
    return S
```

This operation chooses no path, no embedding and no tentative fixed coloring. Both rejections come from actual parity contradictions. Part II proves that no satisfiable instance is rejected.

5 Trim pseudocode

The operation TRIM deletes the returns to depth c.

```
function TRIM(S, c)
    if S is empty then return empty
    if S.a = c then return empty
                                  ▷ otherwise S.a < c, and all live depths are at most c
    while S.P is not empty and min(TOP(S.P)) = c do
        POP(S.P)
    if S.P is not empty then
        Q ← TOP(S.P)
        remove all first nodes of Q.L whose depth is c
        remove all first nodes of Q.R whose depth is c
        update the last pointer of an emptied list to null
    return S
```

All nodes in a popped block have depth c. At most one block can be partly trimmed, and that block keeps at least one node of smaller depth. The anchor is unchanged unless the entire summary becomes empty.

6 Main loop

We process the vertices in the postorder of the search. At a vertex v, we start with S empty and process the outgoing edges of v in the order of their keys:

- for a back edge (v, u), let T be the one-node summary $\big(h(u), [(v, u)], \text{empty block stack}\big)$;
- for a tree edge (v, w), let T be the summary of w computed earlier;
- set $S \leftarrow \textsc{Merge}(S, T)$.

After all outgoing edges of v, if v has a parent u, we store $\textsc{Trim}(S, h(u))$ as the summary of v. A root has an empty final summary. If no merge rejected, we return true. The empty graph and

isolated vertices are accepted by the same code.

Part II

Complete proof of the constraint solver

This part proves that the algorithm of Part I accepts exactly when the fork constraints and weak alignments of Part I, Section 2 can be satisfied together. The idea is that a summary is a normal form of a Boolean formula. It describes exactly which colorings of the back edges that are still active extend to colorings of all back edges below that satisfy every constraint seen so far.

Representation refinement

In this part a logical stack includes a virtual bottom block $(A, \varnothing)$ for the anchor. The physical representation of Part I stores the anchor separately: the logical stack of (a, A, P) is the virtual anchor followed by the physical blocks of P. Every reference below to the bottom block means this logical stack. MERGE never pops the virtual anchor, because $a \leq b$. It concatenates anchors directly when the minima agree. TRIM removes the virtual anchor only when every active depth equals the trimmed depth. Thus one-sided logical blocks and physical summary objects agree exactly, and virtual anchors add no physical pops.

Definitions

A back edge f is a variable with key $d(f)$, the depth of its head. For an outgoing edge e of v, the set $F(e)$ consists of the back edges in the branch of e whose head is a strict ancestor of v, as in Part I, Section 1. Recall that $\mathrm{low}(e)$ is the least key in $F(e)$, with $h(v)$ as sentinel when $F(e)$ is empty, and that e is chordal when $F(e)$ has at least two distinct keys. The outgoing edges are ordered by $(\mathrm{low}, \mathrm{chordal})$, so a thin edge comes before a chordal edge at equal lowpoint. Edges with empty $F(e)$ can be skipped in the constraint phase.

For a tree edge (u, v), the set $W(u, v)$ is the union of $F(e)$ over the outgoing edges e of v. These are the weak returns from $\mathrm{subtree}(v)$, with head depth at most $h(u)$. Weak alignment says that all members of $W(u, v)$ with minimum key have the same color. It is imposed before the key $h(u)$ is deleted.

A block (L, R) represents the formula: L is monochromatic, R is monochromatic, and the colors of L and R differ when both are nonempty. Thus a one-sided block is a freely colored monochromatic class, and every nonempty block has exactly two colorings. There are no constraints between different blocks. The lists L and R are singly linked and ordered by nonincreasing key.

The invariant

At a vertex v, the *active* variables after a prefix of the outgoing branches has been processed are the members of $F(e)$ for the processed branches e. The stack invariant after processing such a prefix is the following.

(I1) Its lists are disjoint and cover exactly the active variables.

(I2) Its block formula is *exactly* the existential projection, onto the active variables, of the conjunction of: all fork constraints at vertices inside the processed branches, all completed weak alignments (those of the tree edges in the processed branches), all fork constraints at v between processed outgoing edges, and the weak alignment of the current minimum-key active variables.

(I3) If a block P is below a block Q, every key of P is at most every key of Q. The inequality is not strict.

(I4) When the stack is nonempty, its unique bottom block is one-sided and contains exactly all minimum-key active variables. No other block contains that minimum. We call it the anchor.

The minimum key of every nonempty prefix is the minimum a over all nonempty outgoing branches, because the branches are processed by increasing lowpoint. Therefore the partial alignment in (I2) is part of the final weak alignment of the incoming tree edge of v, not an extra constraint.

Why completed formulas compose

Back-edge variables of different outgoing branches are disjoint. All constraints at v involve only strict returns of those branches, and the alignment at v involves only the same active variables. So the hidden variables of a branch, which were projected away, never occur in new constraints. Formally, let X and Y be disjoint sets of hidden variables, let Φ_{old} be a formula on X and the old active variables, let Φ_{child} be a formula on Y and the child's active variables, and let Ψ be a formula on active variables only. Then

$$(\exists X, Y.\ \Phi_{\text{old}} \wedge \Phi_{\text{child}} \wedge \Psi) \iff (\exists X.\ \Phi_{\text{old}}) \wedge (\exists Y.\ \Phi_{\text{child}}) \wedge \Psi,$$

because X occurs only in the first conjunct and Y only in the second. This lets us work only with the two block normal forms.

Exact aggregation of new fork constraints

Suppose that e_i is the new branch, with $i \geq 2$, that a is the lowpoint of the first branch e_1, and that $b = \text{low}(e_i)$. Thus $a \leq \text{low}(e_j) \leq b$ for $j < i$. Define

$$B = \{f \in F(e_i) :\ d(f) > a\},$$
$$U = \Big\{f \in \textstyle\bigcup_{j<i} F(e_j) :\ d(f) > b\Big\}.$$

The newly added fork constraints, together with the earlier constraints, are equivalent to:

$$B \text{ monochromatic}, \qquad U \text{ monochromatic}, \qquad B \text{ opposite } U.$$

Here an empty set has its usual vacuous meaning.

Proof. For each $j < i$, write $M_j = \{f \in F(e_j) : d(f) > b\} = I(e_j, e_i)$ and $N_j = \{f \in F(e_i) : d(f) > \text{low}(e_j)\} = I(e_i, e_j)$. The new fork constraints are: M_j is monochromatic, N_j is monochromatic, and M_j is opposite N_j. Since the first lowpoint is a, we have $N_1 = B$, so the fork constraints force B to be monochromatic.

If M_j is nonempty, then N_j is nonempty. Indeed, if $\text{low}(e_j) < b$, every return of e_i belongs to N_j. If $\text{low}(e_j) = b$, then a nonempty M_j means that e_j is chordal, and the order forces e_i to be chordal too, so e_i has a return strictly above b. Also $N_j \subseteq B$, since $\text{low}(e_j) \geq a$. Consequently every nonempty M_j is opposite B, which proves that all of U is monochromatic and opposite B.

Conversely, $M_j \subseteq U$ and $N_j \subseteq B$, so the three aggregated conditions imply every new fork constraint. This proves the equivalence. It also shows that a nonempty U forces B to be nonempty. □

Child half of merge

If $b = a$, we detach the child's anchor, which contains exactly its key-a variables, and append its single list to the old key-a anchor. This imposes exactly the new weak alignment equalities. The detached anchor is independent of every other child block, by (I2) and (I4). If $b > a$, we detach nothing. In either case, the remaining child variables are exactly B.

Every remaining child block must be one-sided. A two-sided block already requires opposite colors, so it cannot satisfy "B monochromatic". If a two-sided block occurs, rejection is therefore exact. Otherwise, we concatenate the nonempty lists of these blocks from top to bottom into one list R. Because the key ranges of the blocks are ordered by (I3), the list R is still nonincreasing. It contains exactly B and represents exactly the old child constraints together with "B monochromatic". Every key in R is at least b, and strictly greater than b when $b = a$.

Old half of merge

We pop old blocks while their maximum key is greater than b. These are exactly the old blocks that meet U: once a block with maximum at most b is reached, (I3) implies that all lower blocks also have maximum at most b.

For each popped block:

- If both sides have a member of U, we reject. The existing constraint that the two sides have opposite colors contradicts "U monochromatic".
- Otherwise exactly one side meets U. We swap the sides if needed, so that it is L. Its entire list must have the common color of U, because that side was already monochromatic. Its opposite list R_{old} has the opposite color, and hence the color of B, since a nonempty U forces B to be nonempty.

We concatenate these L lists from top to bottom into a new list L. We append each nonempty opposite list after the new child list R. We push (L, R) if it is nonempty.

This encodes *exactly* the old block constraints together with "U monochromatic" and "B opposite U". Every side of an affected block is forced into the stated class, and an unaffected block has no variable of U and stays independent. No arbitrary extra equality is added.

List ordering and stack ordering

The new list L is nonincreasing, because the old blocks were popped from higher keys to lower ones. Every old opposite list has all keys at most b, whereas every key of the child list R is at least b, so appending keeps the order. In fact at most the *lowest* popped block can have a nonempty opposite side. If an upper popped block had a key at most b, then (I3) would force every key of a lower block to be at most b, contradicting the fact that the lower block was popped. So opposite lists are never interleaved.

When U is empty, every old key is at most b. So the new one-sided block B, whose minimum is at least b, can be pushed above all old blocks. When U is nonempty, every key left in the old stack is at most b, by the stopping condition, and at most every key of every popped block, by (I3). Hence it is at most every key of B and every old key put in the new block. This proves (I3) after the push.

The old anchor has only the key $a \leq b$, and it is never popped. All variables outside it have key greater than a, by (I4), and the child's key-a anchor was appended to it when necessary. Therefore (I4) is preserved, including the uniqueness of the minimum-key anchor.

All list ownership is transferred, not copied. So (I1) follows from the disjointness of the variables of different branches and from the transfer of whole blocks.

Trim at the parent

After all branches of v have been processed, every active return has head key at most $h(u)$, where u is the parent of v. We delete every variable with key $h(u)$. Because of (I3), we first pop all entire blocks whose minimum is that key. Then at most one remaining block can contain that key, and in each of its lists these variables form a prefix. We delete the prefixes, and delete the block if both sides become empty.

This is an *exact* existential projection of a block formula. To extend a coloring of the survivors, give each deleted variable the color of the survivors on its side. If one side or both sides have no survivors, give the missing side the color opposite to the other side, or an arbitrary color when no survivor remains. Thus (I2) is preserved, and now the weak alignment of (u, v) is a completed alignment. The remaining variables are exactly $F((u, v))$. Ordered lists and the stack order persist under deletion. If anything remains, the old minimum a is less than $h(u)$, so its untouched anchor is still the unique minimum anchor. Otherwise the stack is empty.

Base and completion

A direct outgoing back edge gives a one-sided block with one variable. It has no internal constraints and satisfies all invariants. A processed tree edge to a child supplies the invariant by induction. At a root of the search there are no strict returns, so all variables have been existentially eliminated. A successful empty normal form therefore means that all constraints in that component have a satisfying assignment. Rejection happens exactly when an explicitly required equality would identify two opposite nonempty classes. By Part III and Part IV, the algorithm accepts exactly when the graph is planar.

Linear work

We make one list node per back edge, and singly linked stacks of block records. We store list heads and tails, block minima and maxima, and anchor pointers. A one-edge summary creates one block. Each merge of an outgoing edge creates at most one new block and permanently consumes every popped block of the child and of the old stack. A discarded anchor block is consumed as well. Thus $O(m + n)$ blocks are created and $O(m + n)$ block operations occur, where n and m are the numbers of vertices and edges. Each active back-edge node is deleted once, so the total work of Trim is $O(m)$. Constant-time pointer concatenations never scan a list. With arrays, explicit search frames and the integer-key counting sort, the bound $O(n + m)$ is deterministic on a word RAM.

Part III

Weak alignment does not change satisfiability

The algorithm decides the fork constraints together with all weak alignments. This part shows that adding the weak alignments does not change whether the fork constraints can be satisfied.

Let $p = (u, v)$ be a tree edge, let $W(p)$ be its set of weak returns, and let $M(p)$ be the set of its members of minimum depth ℓ. Assume that $W(p)$ is nonempty. We read the fork constraints as pairwise equalities within each nonempty set $I(e, f)$ and pairwise inequalities between the two nonempty sets. Only distinct variables give a nontrivial equality. We call a variable *isolated* if it occurs in no nontrivial equality or inequality, and *nonisolated* otherwise.

Suppose that $b \in M(p)$ occurs in one of these equalities or inequalities, at a fork w. For b to

belong to $I(e, f)$, the other branch f must have lowpoint below ℓ. Consequently f has a back edge with head depth below ℓ. The vertex w lies on the tree path from the root to the tail of b. It cannot belong to subtree(v). If it did, the back edge of f would have its tail in subtree(v) and its head depth below $\ell \leq h(u)$, contradicting the choice of ℓ as the minimum depth in $W(p)$.

Therefore w is a strict ancestor of v. All members of $M(p)$ then belong to the same outgoing branch e of w, the one containing v, and have the same head depth ℓ. So they all belong to the same set $I(e, f)$, and the original constraints already force them to have one color.

It follows that each $M(p)$ either is already forced to be monochromatic by the original constraints, or consists entirely of variables that occur in no nontrivial original equality or inequality. To satisfy all alignments at once, we keep a satisfying coloring of every nonisolated variable and give every isolated variable the color R. A set of the first kind is already monochromatic, and a set of the second kind now is. The fork constraints still hold, since no nonisolated variable changed its color. Thus the original fork constraints can be satisfied if and only if the fork constraints together with *all* weak alignments can be satisfied.

Part IV

The planarity characterization, proved constructively

This part proves that the constraints of Part I, Section 2 can be satisfied exactly when the graph is planar. We first show that a plane embedding gives a coloring that satisfies the fork constraints. Then Sections 1 to 5 show that a coloring that satisfies the fork constraints and the weak alignments gives an explicit plane drawing. Sections 6 and 7 supply the step from a plane embedding to a nonalternating contour, which the first argument assumes.

Contours and nonalternation. A *rotation* at a vertex is a cyclic order of its incident edges. Given a rotation at every vertex, we build a word of tokens, the *contour*. At a nonroot vertex, we cut its rotation immediately after the parent incidence, so that it becomes a linear order. At the root, we cut at any corner. Then we replace each outgoing tree incidence, recursively, by the contour of the child. A back edge (x, u) has a *source token* at x and a *target token* at u. Two back edges *alternate* if exactly one token of the second lies strictly between the two tokens of the first. A contour is *nonalternating* if no two back edges alternate in it.

The fork constraints are necessary. Assume that the rotations of a plane embedding give a nonalternating contour. Sections 6 and 7 prove this assumption. We declare a back edge (x, u) to be L if its target token occurs before the child incidence at u leading to x, and R otherwise.

Take two back edges $b_1 = (x_1, u_1)$ and $b_2 = (x_2, u_2)$ whose fundamental tree paths share an edge, with $h(u_1) > h(u_2)$. The *fundamental tree path* of a back edge (x, u) is the tree path from u to x. Let w be the lowest common ancestor of x_1 and x_2, and let e_1, e_2 be the distinct outgoing edges of w that lead to the respective source tokens. Such an edge can itself be an outgoing back edge, when its source is w. Both sources lie in the same child subtree of u_1. The target token of b_1 occurs before both source tokens when b_1 is L, and after both when it is R. The target token of b_2 is outside the entire interval of the subtree of u_1. Checking the order of these four tokens, nonalternation implies:

$$e_1 \text{ precedes } e_2 \text{ at } w \quad \text{if and only if} \quad b_1 \text{ is L.} \tag{1}$$

Now take distinct outgoing edges e, f of a vertex w. For any $b \in I(e, f)$, choose a return $c \in F(f)$ of minimum depth. It exists because $d(b) > \text{low}(f)$, whereas $d(b) < h(w)$. The two fundamental

paths share the incoming edge of w, and the head of b is deeper than the head of c. So (1) gives the same color to every member of $I(e, f)$, determined only by whether e precedes f. Reversing the roles gives the other color to every member of $I(f, e)$. These are exactly the fork constraints.

Thus a plane embedding gives a coloring that satisfies the fork constraints, and Part III makes it weakly aligned. The following construction proves the converse in full.

1 Definitions

Root the search tree T. All non-tree edges $b = (x, u)$ go from a descendant x to a strict ancestor u, and $h(v)$ is the depth. For an outgoing edge e of v, the set $F(e)$ of its returns and the values $\mathrm{low}(e)$, $\mathrm{chordal}(e)$ and $\mathrm{key}(e) = 2\,\mathrm{low}(e) + \mathrm{chordal}(e)$ are as in Part I, Section 1. When $F(e)$ is nonempty, let $\mathrm{high}(e)$ be the maximum head depth in $F(e)$.

We assume a coloring with the following properties. For each fork at a nonroot vertex v, and distinct outgoing edges e, f of v:

- the set $\{b \in F(e) : d(b) > \mathrm{low}(f)\}$ is monochromatic;
- the analogous set with e and f exchanged is monochromatic;
- if both sets are nonempty, their colors are opposite.

Moreover the coloring is weakly aligned: for every tree edge (p, v), among the back edges with source in $\mathrm{subtree}(v)$ and head equal to p or an ancestor of p, all those of minimum head depth have the same color. If this set is empty there is no condition. The inclusion of p is deliberate. It covers the alignment just before the stack algorithm removes the returns to p.

We extend the colors to outgoing edges. For an outgoing tree edge e with $F(e)$ nonempty, let $\sigma(e)$ be the color of any return of depth $\mathrm{high}(e)$. If $F(e)$ is empty, choose any color. For a back edge, $\sigma(e)$ is its given color.

We order the outgoing edges at v by putting all L edges first, in decreasing key, and then all R edges, in increasing key. Ties are broken arbitrarily. Every comparison used below is strict, so the tie breaking does not matter.

We run a depth-first enumeration of these ordered outgoing edges, and give every back edge a distinct *source rank* when its outgoing occurrence is reached. Incoming back-edge occurrences do not enter this enumeration.

2 An explicit contour word and rotation

At a vertex v, each outgoing tree edge $e = (v, w)$ receives two lists:

$$L_e = \text{all back edges } (x, v) \text{ with } x \in \mathrm{subtree}(w), \text{ colored } \mathsf{L},$$
$$R_e = \text{the corresponding } \mathsf{R} \text{ edges.}$$

In each list, we order the edges by decreasing source rank.

We replace each outgoing tree edge $e = (v, w)$ by the block

$$\text{target ports } L_e; \quad \text{child } w; \quad \text{target ports } R_e,$$

and each outgoing back edge b by its source port. At nonroot vertices, the incoming tree edge precedes this list in the cyclic rotation. These rules give a rotation at every original vertex.

Equivalently, add a distinct leaf of degree one for every source port and every target port. The resulting rooted ordered tree T^* has a leaf contour word W. Inserting target ports does not

change the order of the source ports, so the source ranks used to build the lists are exactly the order of the source ports in W.

For a tree edge $e = (v, w)$, its whole block $L_e\, W(w)\, R_e$ is contiguous, where $W(w)$ is the contour of the subtree of w. More generally, $W(w)$ is a contiguous subword of W for every rooted subtree. At the lowest common ancestor v of two sources, the order of their source ports is the order of their distinct outgoing fork edges at v. If a source is v itself, we use the corresponding back edge as that outgoing fork edge.

3 The fork-order lemma

Consider back edges $b_1 = (x_1, u_1)$ and $b_2 = (x_2, u_2)$ whose tree paths share an edge. Let v be the lowest common ancestor of x_1 and x_2, and let e_1, e_2 be their distinct outgoing fork edges at v. Both u_1 and u_2 are strict ancestors of v.

Lemma (A). *Suppose that $h(u_1) > h(u_2)$. Then e_1 precedes e_2 in the outgoing order if and only if b_1 is* L.

Proof. Since $h(u_1) > h(u_2) \geq \mathrm{low}(e_2)$, the first fork constraint makes all returns of e_1 above $\mathrm{low}(e_2)$ have the color of b_1. In particular its highest return does, so $\sigma(e_1)$ is the color of b_1. If $\sigma(e_2)$ has the other color, the conclusion follows from the rule that L edges come before R edges. Otherwise both edge colors are the same. Then the two opposite fork sets cannot both contain their highest returns, so

$$\mathrm{high}(e_2) \leq \mathrm{low}(e_1).$$

Consequently $\mathrm{low}(e_2) \leq \mathrm{high}(e_2) \leq \mathrm{low}(e_1)$. If $\mathrm{low}(e_2) < \mathrm{low}(e_1)$, then $\mathrm{key}(e_2) < \mathrm{key}(e_1)$. If the lowpoints are equal, every return of e_2 is at that common lowpoint, whereas b_1 returns strictly higher. Thus e_2 is not chordal and e_1 is chordal, which again gives $\mathrm{key}(e_2) < \mathrm{key}(e_1)$. The signed order therefore puts e_1 first exactly when b_1 is L. □

Lemma (B). *Suppose that $u_1 = u_2 = u$, that b_1 is* L *and that b_2 is* R*. Then e_1 precedes e_2.*

Proof. Put $l_i = \mathrm{low}(e_i)$, so $l_i \leq h(u)$.

- If both $l_i < h(u)$, the fork constraints force $\sigma(e_1) =$ L and $\sigma(e_2) =$ R.
- If $l_1 < h(u) = l_2$, they force $\sigma(e_2) =$ R, and $\mathrm{key}(e_1) < \mathrm{key}(e_2)$. Whether $\sigma(e_1)$ is L or R, the edge e_1 precedes e_2.
- If $l_2 < h(u) = l_1$, then symmetrically $\sigma(e_1) =$ L and $\mathrm{key}(e_2) < \mathrm{key}(e_1)$, so again e_1 precedes e_2.
- Suppose that $l_1 = l_2 = h(u)$. Let (p, v) be the incoming tree edge of v. If its weak minimum head depth were $h(u)$, alignment would force b_1 and b_2 to have the same color. Therefore that weak minimum is strictly below $h(u)$. Since $h(p) \geq h(u)$, both b_1 and b_2 are weak returns of (p, v), so some third outgoing edge e_0 of v has $\mathrm{low}(e_0) < h(u)$. The fork constraints with e_0 then make every return of e_i have the color of b_i, including its highest return. Thus $\sigma(e_1) =$ L and $\sigma(e_2) =$ R, which proves the claim. □

4 No two back-edge port pairs alternate in W

Let $b_i = (x_i, u_i)$, for $i = 1, 2$, be distinct back edges.

Paths that share no edge. If their fundamental tree paths share no tree edge, their port pairs do not alternate. Here are the complete structural cases. If the heads are incomparable in the rooted tree, both pairs occur in disjoint subtree words. Otherwise we may assume that u_1 is an ancestor of u_2. If x_1 lies outside subtree(u_2), both ports of b_2 lie in $W(u_2)$, which contains neither port of b_1. If x_1 lies in that subtree, but the two paths share no edge, then either their sources diverge at u_2 or $x_1 = u_2$. The ports of b_2 both lie in the single outgoing child block of u_2 leading to x_2. The source port of b_1 is outside that block, and so is its target port. If $u_1 = u_2$ and the sources diverge immediately, the pairs lie in disjoint outgoing child blocks. In all cases one pair is in a contiguous subword that contains neither port of the other, which excludes alternation.

Paths that share an edge. Now suppose that their paths share an edge. The heads are comparable, and both are strict ancestors of the source fork v.

First suppose that $h(u_1) > h(u_2)$. The target port of b_2 lies outside $W(u_1)$. The target port of b_1 lies on the L side or the R side of the child block of u_1 that contains v. Both source ports lie inside that block. If b_1 is L, its target comes before both source ports, and alternation would require the source of b_2 to come before the source of b_1. If b_1 is R, its target comes after both source ports, and alternation would require the source of b_1 to come before the source of b_2. Both possibilities contradict Lemma (A), since the order of the source ports is the order of e_1 and e_2 at v.

Finally suppose that $u_1 = u_2$. If their colors agree, their target ports are in one list on the same side, ordered in reverse order of their source ports. Their four ports are therefore nested, not alternating. If their colors differ, say b_1 is L and b_2 is R, the target of b_1 precedes the shared child block and the target of b_2 follows it. Alternation would require the source of b_2 to come before the source of b_1, contrary to Lemma (B). This exhausts all cases.

5 Convert a nonalternating contour to explicit curves

This step applies to any finite rooted ordered tree with a matching on some of its leaves whose pairs do not alternate in the leaf order. We apply it to the tree T^* of Section 2, where the matching pairs the two ports of each back edge.

Add one dummy leaf to each original vertex which otherwise has no child or port leaf. All leaves are now genuine ports or dummy ports. Let D be their maximum depth. Subdivide each leaf edge as necessary so that every leaf has depth D. Subdivision does not change the leaf order. Number all leaves $1, \ldots, r$ in order.

For an internal vertex z, let $L(z)$ and $R(z)$ be the smallest and largest indices of its descendant leaves, and put

$$\mathrm{pos}(z) = \Big(\frac{L(z) + R(z)}{2},\ D - \mathrm{depth}(z)\Big).$$

Put leaf i at $(i, 0)$, and draw every tree edge as a straight segment.

This is a plane tree drawing. The descendant-leaf intervals of distinct vertices at the same depth are disjoint. Hence edges from distinct parents in one level strip lie in disjoint horizontal intervals. Edges from the same parent end at distinct x-coordinates on the next horizontal level, and their straight segments meet only at the parent. Distinct open level strips are disjoint. Their common boundary lines meet edges only at endpoints, and distinct vertices on one such line have distinct x-coordinates. Thus tree-edge images meet only at common endpoints. Every original vertex has a descendant leaf at strictly larger depth, so all original vertices lie strictly above the line $y = 0$.

The back edges. For each matched pair of genuine port leaves $i < j$, draw the polygonal U-shaped path with three segments

$$(i, 0), \quad (i, -(j-i)), \quad (j, -(j-i)), \quad (j, 0).$$

Distinct matching pairs have four distinct endpoints. Nonalternation says that their intervals are either disjoint or one is strictly inside the other. Disjoint intervals give disjoint horizontal ranges for the two U paths. If $[k, l]$ is strictly inside $[i, j]$, the smaller U lies strictly between the vertical sides of the larger U and strictly above its bottom, since $l - k < j - i$. Thus the U paths are disjoint. All their interior points have $y < 0$, so they also miss the tree drawing, which lies in $y \geq 0$. Every coordinate is rational.

Each original non-tree edge is now represented by its source-port path, its lower U path, and its target-port path. These three pieces form a simple curve between the original endpoints. Port paths for distinct ports are disjoint except when they are attached at the same original vertex, and the U pieces are pairwise disjoint. Consequently distinct original edges meet exactly when they have a common endpoint, and then only there. Delete the dummy port paths and suppress all port vertices and subdivision vertices. This is an explicit plane drawing of the original graph.

For a disconnected graph, construct a drawing of each component independently and place translated and scaled copies in disjoint rectangles.

6 Optional necessity by face permutations

Suppose that a plane embedding is given combinatorially, as a rotation system whose number of faces satisfies Euler's formula with characteristic 2. Then the step from the embedding to a nonalternating contour needs no topology. We treat a connected graph. Contract the tree edges one at a time. In a dart rotation, contracting an edge between distinct vertices deletes its two darts and splices the two vertex cycles. For the face permutation $\varphi = \text{rotation} \circ \text{reversal}$, this only deletes those darts from their face cycles, so the number of faces is preserved. The single exceptional component with no darts counts as having one face.

After all tree edges are contracted, the remaining rotation is exactly the contour word W, with b matched pairs, one vertex, and $b + 1$ faces by Euler's formula. If $b = 0$, the word W is empty and nonalternation holds trivially. Assume that $b > 0$. If W has no cyclically adjacent matched pair, then φ has no cycle of length one, so it has at most $2b/2 = b$ cycles, a contradiction. Delete a cyclically adjacent matched pair. One of its darts was a face of length one, and deleting both darts decreases the number of faces by one. For $b = 1$, the two faces of length one become the one empty face. By induction, all pairs can be deleted in this way. An adjacent pair cannot alternate with any other pair, and deletion preserves alternation among the remaining pairs. Therefore W is nonalternating.

This argument uses the exact Euler identity for rotation systems, not just the inequality $m \leq 3n-6$. For embeddings given geometrically, Section 7 proves the same step from the edge bound alone.

7 Geometric necessity using only the Euler edge bound

If the available fact from Euler is only $m \leq 3n - 6$, the following argument gives the step from a plane drawing to a nonalternating contour. It uses elementary compactness and continuity, and finite polygonal paths. The topology of a tree neighborhood is spelled out in Section 7b. We treat one component of the graph and its search tree. For a disconnected graph, we apply the argument to each component.

7a Polygonalize a finite continuous plane drawing

Parameterize every edge image by an injective continuous arc $\gamma\colon [0,1] \to \mathbb{R}^2$. At each vertex u, choose a small closed disk D_u centered at u. We choose these disks pairwise disjoint, and we require D_u to avoid every edge image not incident to u. For an incident edge parameterized from u, we also require D_u to avoid $\gamma([1/3,1])$. These requirements can be met, because they concern finitely many compact sets that do not contain u. For γ from u to v, let t_u be the last parameter at which γ lies on the boundary of D_u, and t_v the first parameter at which it lies on the boundary of D_v. Since the same edge parameterized from v is $t \mapsto \gamma(1-t)$, the disk D_v avoids $\gamma([0,2/3])$. Continuity gives $t_u < 1/3 < 2/3 < t_v$, and the middle arc $\gamma([t_u, t_v])$ lies strictly outside the two disks except at its ends. It avoids all other vertex disks as well.

Replace each end part by the radial segment from its vertex to its exit port. Radial segments of different incident edges are disjoint except at their vertex, because their exit ports are distinct points of one circle. Every middle arc is disjoint from all radial segments of other edges. The middle arcs are pairwise disjoint compact sets. Therefore we can choose for each middle arc a small open metric neighborhood, so that these neighborhoods are pairwise disjoint and avoid all radial segments of other edges and every original vertex. Uniform continuity of the arc gives a finite partition whose consecutive sample points can be joined by straight segments inside this neighborhood: choose the samples so close that each such segment stays inside a small ball about one sample point.

Append the two radial segments of the edge. This gives a finite polygonal walk between the original ends, which misses every other constructed edge walk except at shared original ends. To remove self-intersections, split its segments at all isolated pairwise intersections and at the endpoints of collinear overlaps, identify coincident subsegments, and take a simple path in the resulting finite arrangement of segments. This is a simple polygonal arc between the same ends. Doing this for each edge gives a polygonal plane drawing. The argument uses neither the Jordan curve theorem nor the Schoenflies theorem.

7b A thin neighborhood of a polygonal embedded tree is a disk

Subdivide the drawn tree at each bend, without changing its image. Around each tree vertex, choose a small convex polygon. It has one facet perpendicular to each incident edge direction, and the edge crosses the relative interior of that facet. Such a polygon can be made by intersecting finitely many half-planes tangent to a small circle, including one outward normal in each required direction. Extra normals can be added to make the intersection bounded. Distinct incident edges have distinct directions near their common endpoint. We make the polygons disjoint, and small enough to avoid all edges not incident to their vertex and all later segments of incident edges. Along each straight tree segment between its end polygons, we place a thin closed rectangular strip. It meets each end polygon exactly in a short boundary interval of the right facet. Otherwise all strips and polygons have disjoint interiors and no unintended meetings. Finitely many positive separation distances allow all widths to be chosen. The strip intervals on one polygon are pairwise disjoint.

The union N is a closed disk. Here is an elementary gluing proof. Every convex polygon is a disk, by radial parametrization. The union of two disks that meet in exactly one nondegenerate proper closed boundary interval is a disk. To see this, use the given disk parametrizations to map the two disks to the two halves of a square, with the gluing interval going to the common side. Choose boundary homeomorphisms that agree pointwise on the common interval, and extend them over the complementary boundary arcs. Every homeomorphism f of the circle extends explicitly over the unit disk by $H(0) = 0$ and $H(rz) = rf(z)$ for $|z| = 1$ and $0 < r \leq 1$. Its inverse is the radial extension of f^{-1}. Radial parametrizations of the convex half-squares give

the same construction for those domains. Thus the two disk maps can be chosen to agree on the common side. Glue the half-squares to one square. The induced map from that square to the union is continuous and bijective, hence a homeomorphism, because its domain is compact and the plane is Hausdorff. The outer boundary of the square maps to the boundary formed by the two old boundaries without the interior of the glued interval. This boundary property is kept at each attachment. For the unions of polygons and strips here, this is also their boundary in the plane, as one checks locally at points of a facet, of a corner, or of a glued interval. Starting from one vertex polygon, we attach one tree strip and then its new vertex polygon at a time. Each attachment meets the previous union in exactly one boundary interval, so the gluing lemma applies by induction.

All images of non-tree edges, except their end tails, lie outside N, and each meets the boundary exactly once at each end. Traversing the boundary of N visits these ports in exactly the contour order given by the cyclic orders of the edges at each original vertex. This follows by induction from the same attachment of leaves: a strip substitutes the boundary contour of the child for the interval where it was attached, which is exactly the recursion of the rooted-tree contour.

7c Alternating exterior arcs would give a plane K_5

Suppose that two non-tree edges have alternating boundary ports p_1, p_2, p_3, p_4, with exterior arcs joining p_1 to p_3 and p_2 to p_4. Use the four boundary arcs between consecutive ports as a 4-cycle. A homeomorphism from the unit disk onto N gives one interior point z and four internally disjoint arcs from z to the four ports: the images of the four radial segments. These four spokes are inside N, and the two exterior arcs are outside N. Consequently the cycle, the four spokes and the two exterior arcs form a plane drawing of K_5. It has five vertices and ten edges, contradicting Euler's bound $10 \le 3 \cdot 5 - 6 = 9$. Therefore no two pairs of ports of non-tree edges alternate.

This proves that a plane drawing gives a nonalternating tree contour, using only the Euler edge bound as a fact about planar graphs. Section 5 proves the reverse implication by explicit coordinates. So the proof works both with geometric embeddings and with rotation systems, and no fast planarity algorithm is assumed in either direction.

Part V

Word RAM bound and use in the crossing extractor

This part shows that the algorithm is one fixed word-RAM program with a linear bound, collects the correctness proof, and explains how the crossing extractor uses it.

1 A single fixed program

We use vertex labels $0, \dots, N-1$, edge identifiers $0, \dots, M-1$, and null pointers. All working structures are arrays of records of fixed size: search frames, depth and parent arrays, two lowpoint arrays, bucket links, links of outgoing lists, back-edge nodes, summary records, and block records. Fields contain only indices, depths in $[0, N]$, or Boolean values. Array capacities are allocated from N and M. If M is not part of the input header, we first count the adjacency entries in time $O(N + M)$. The word size need only address $O(N + M + 1)$ words.

Every line of the pseudocode expands into a fixed number of RAM operations: read or write a word, compare, branch, add or subtract an index, or move a pointer. The multiplicative factor

does not depend on the input graph. Appending to a linked list changes at most a constant number of fields. No implicit copying of a list or a stack is permitted or needed.

2 Total operations

The first search scans $2M$ adjacency entries and has N vertex-completion steps. Every edge causes at most two lowpoint insertions into the record of its parent edge. The global bucket pass scans $O(N)$ buckets and M edge records. The postorder constraint pass visits N vertices and M outgoing-edge records.

One return-list node is allocated per back edge, so at most M nodes in total. Only one-edge summaries of back edges allocate summary objects. All returns of child tree edges transfer the ownership of an existing object. Each effective merge creates at most one new block, so there are at most M block creations. Every popped block is permanently consumed: it is never pushed again. Thus all child pop loops, all pop loops of the previous stack and all whole-block trim loops together execute at most M block pops. Every iteration of a partial trim removes one return node permanently, so these iterations total at most M. Dropping a whole anchor or a whole block does not traverse its nodes. Each tree edge has one call of Trim, with at most one partly trimmed block.

Consequently the total number of RAM instructions is bounded by

$$C\,(N + M + 1)$$

for a fixed constant C, and the total number of allocated words has the same form. This is a deterministic worst-case bound. It assumes neither average-case hashing nor amortized union-find operations. The charging argument for linked lists above bounds the sum of the operations in one run, in the worst case.

3 Correctness theorem

The two implications now have explicit finite chains of reasons.

If the graph has a plane embedding, Part IV, Section 7 (or the rotation proof in Section 6) gives a nonalternating contour for the search tree. The fork comparison at the beginning of Part IV gives a coloring that satisfies every fork constraint. Part III gives a weakly aligned one. The exact projection invariant of Part II implies that no merge rejects. Hence the program returns true.

If the program returns true, the final empty summary of every component is a satisfiable existential projection. Reversing the finite projections and merges, using their extension properties proved in Part II, gives a full aligned coloring. Part IV, Sections 2 to 4 produce a nonalternating contour. Section 5 gives an explicit rational polygonal drawing of that component. We put distinct components in disjoint translated and scaled rectangles. Thus the graph has a plane embedding.

The coloring and the drawing are witnesses used only in the correctness proof. The Boolean program does not construct them, so their construction costs nothing in the linear bound.

4 Use in the crossing extractor

The crossing extractor of the companion note [HJ26] calls a Boolean planarity test inside its classification step, and states the test as Fact P. The program of this note is such a test, and

the extractor uses it without any change to its classification step. Each planarity call still costs $O(|V(K)| + |E(K)| + 1)$ on the current graph K. All bounds of the companion note on the number and the sizes of the classification calls therefore remain valid, and the crossing extractor still runs in time $O(nm)$.

The planarity program uses one full depth-first search and linear scans. It calls no path search and no flow computation between two vertex sets. Thus it adds no searches to the search count of the companion note, beyond constant work per planarity call.

5 Tests

The file `planarity_linear.py` in this folder implements the program of Part I in Python, with arrays and linked records as in Part I, Section 3. Its function `is_planar` takes the adjacency lists of a simple graph. We compared it with the independent left-right implementation `check_planarity` of NetworkX 3.2.1 [HSS08]. The answers agreed on all 33,868 simple labeled graphs with at most six vertices, including the empty graph, and on 10,000 random graphs with up to 150 vertices and random orders of the adjacency lists, of which 5,910 are planar. These are checks of the implementation. The proof above, not these finite tests, establishes correctness for every input. Python lists are not the arrays of a word RAM, so the implementation illustrates the program but does not prove its time bound.

Part VI

The Lean formalization

The Lean formalization of this note is in the repository `GraphMinor`. A plane embedding is the structure `HP25.PlaneEmbedding` of the formalization of Humeau and Pous: the vertices are distinct points of the complex plane, and the edges are arcs that meet only at common ends.

- The folder `GraphMinor/LRPlanarity` proves the mathematics of Parts I to IV. Its file `Statements.lean` lists the statements, each with the place of the note it formalizes. The search and the lowpoints follow Part I, Section 1. The solver follows Part II (`Merge.lean`, `Trim.lean`, `Aggregation.lean`, `Solver.lean`), weak alignment follows Part III (`Alignment.lean`), and the files `ForkOrder.lean`, `Contour*.lean`, `TreeDrawing*.lean`, `Polygonalization*.lean`, `TreeNeighborhood*.lean` and `ExteriorK5*.lean` follow Part IV.
- In the folder `GraphMinor/Compute`, the files `PlanarityRam*.lean` give the word-RAM program and prove its step bound, following Part V, Sections 1 and 2. The file `SpecsPlanarity.lean` states what the program computes.
- The statement `PlanarityTestStatement`, in the file `CitedPlanarity.lean` of the same folder, is Fact P of the companion note. It is proved with no hypotheses, as `planarityTestStatement_proved` in `DisjointPathsFinal.lean`, by assembling the parts as in Part V, Section 3. The same file then proves the main result of Graph Minors XIII, `disjointPathsStatement_proved`.

The Lean formalization differs from this note in the following places.

- *The model of computation.* The programs are fixed word-RAM commands with a logarithmic word size, and every read, write and allocation has a proved bound. A null pointer is zero, an optional index is stored plus one, and pointer arenas reserve slot zero. The step count uses adjacency entries rather than undirected edges, which changes only the constant.
- *The coordinates of Part IV, Section 5.* Leaves are numbered from zero. One leaf word

numbers the leaves of all components, and each component lies in its own vertical strip, instead of in a separate translated and scaled rectangle. The segment to a port is followed by a vertical segment that contains the subdivision vertices and the vertical side of its U path. After the subdivision vertices are suppressed, these are the same drawings.

- *The route to necessity.* The Lean works with geometric embeddings and follows Part IV, Section 7. It does not use the argument of Section 6 for rotation systems.

References

[Br09] U. Brandes, The left-right planarity test, manuscript, Universität Konstanz, 2009.

[dFOMR06] H. de Fraysseix, P. Ossona de Mendez and P. Rosenstiehl, Trémaux trees and planarity, *Internat. J. Found. Comput. Sci.* **17** (2006), 1017–1029. Preprint arXiv:math/0610935v1 (30 October 2006).

[dFR82] H. de Fraysseix and P. Rosenstiehl, A depth-first-search characterization of planarity, *Ann. Discrete Math.* **13** (1982), 75–80.

[dFR85] H. de Fraysseix and P. Rosenstiehl, A characterization of planar graphs by Trémaux orders, *Combinatorica* **5** (1985), 127–135.

[GM13] N. Robertson and P. D. Seymour, Graph Minors. XIII. The disjoint paths problem, *J. Combin. Theory Ser. B* **63** (1995), 65–110.

[HJ26] M. Hajiaghayi and M. JafariRaviz, Extracting a crossing in time $O(nm)$, note, November 2026, file `docs/crossing/crossing_extractor.tex` of the repository `GraphMinor`.

[HSS08] A. A. Hagberg, D. A. Schult and P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008, 11–15.

[HT74] J. Hopcroft and R. Tarjan, Efficient planarity testing, *J. ACM* **21** (1974), 549–568.

[Sh80] Y. Shiloach, A polynomial solution to the undirected two paths problem, *J. ACM* **27** (1980), 445–456.